Joiner workflow walkthrough

JBC Cybersecurity Enterprise IGA Lab

I implemented the joiner workflow to onboard synthetic employees from the PostgreSQL identity store into Okta. I first tested account creation, activation requests and policy-based group assignment with EMP006, Ahsoka Tano, a Security Analyst. I then extended the workflow to the six-person lab population.

This walkthrough follows the connector checks, identity comparison, controlled target changes and verification steps. The captured results show six managed accounts in PROVISIONED status and eleven managed group memberships. EMP006 received three required groups. User setup remained pending; ACTIVE status and successful sign-in were not established.

This project is an independent IGA simulation modeled on common enterprise identity-governance workflows. It was not built in a Saviynt tenant.

Choose a website subsection

Website subsection	Focus	PDF pages
Preflight and account creation	Connector reads, identity matching and staged creation	2-4
Activation	Controlled request and verified pending-setup state	5
Group assignment	Policy evaluation, target verification and safe reruns	6-8
Verification and results	Safety tests, failure handling and remaining evidence gaps	9

The screenshots record prior lab work. I did not rerun the tests for this walkthrough. Missing public evidence is marked beside the relevant step, and remaining checks are listed at the end.

Blake Cox | Joiner workflow portfolio module | Synthetic identities only

Connector validation before provisioning

Step 1 Validate the connector with read operations

I tested authentication and inventory reads before enabling account changes. The validation returned one Okta user and five groups and passed the token-cache check. A later capture returned seven groups as the lab configuration evolved.

Figure 1 shows the Demonstrating Proof of Possession (DPoP) exchange. The first token request received a nonce challenge. The connector generated a new proof and the second request succeeded. A nonce is a server-supplied challenge value used in that proof. User and group reads then returned HTTP 200.

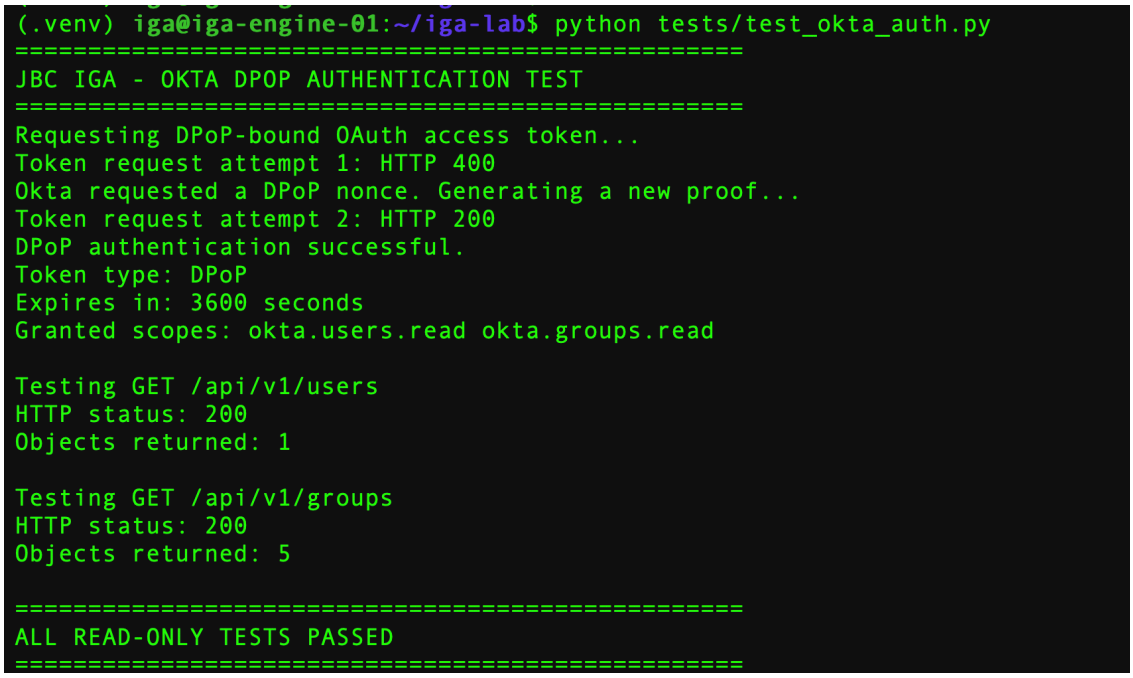


Figure 1. The connector handles the DPoP nonce challenge and successfully reads the Okta user and group inventories.

Step 2 Check permissions before target writes

I checked the service-app scopes and worked through the role configuration. Early captures show okta.users.read and okta.groups.read; a later capture also shows okta.users.manage. The editor screens do not prove the final saved write-role assignment or its resource scope. Successful write execution is verified in later steps.

[SCREENSHOT NEEDED: Final saved service-app role and resource assignment supporting controlled writes, with client IDs, key IDs, tenant details and browser chrome removed.]

Identity comparison and creation preflight

Step 3 Compare HR identities with target accounts

I compared the six authoritative PostgreSQL identities with the Okta inventory. Figure 2 shows zero matched managed accounts and six missing accounts, although the target returned one user. I needed an identity correlation, a reliable link between an HR person and a target account, before treating an existing user as a match.

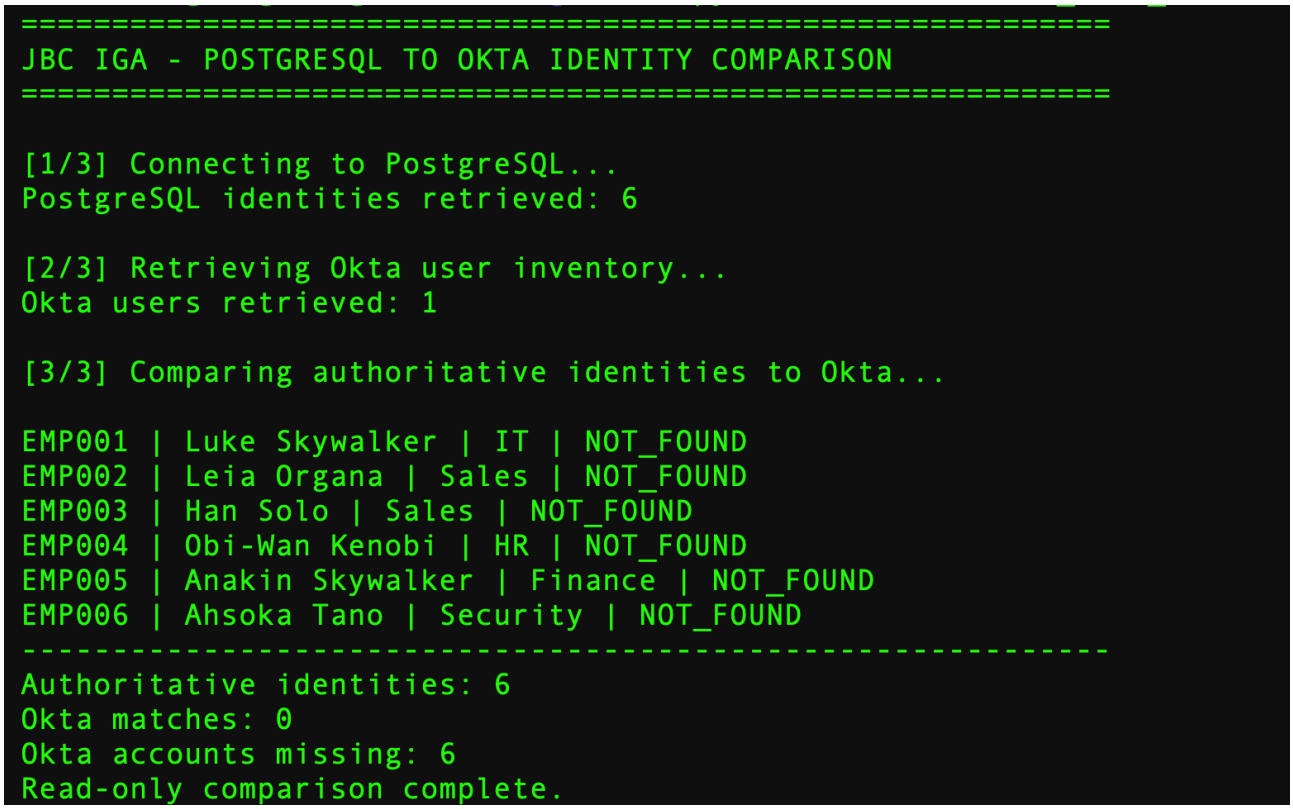


Figure 2. The preprovision comparison identifies six missing managed accounts without treating the existing target user as an HR match.

Step 4 Review the EMP006 creation dry run

I reviewed a payload mapping employeeID to employeeNumber and including the synthetic name, department, title, email and login. The preflight checked the authoritative identity, active status, managed-identity boundaries and existing correlation or login conflicts. It reported zero target modifications and zero PostgreSQL writes.

Staged account creation and reruns

Step 5 Create and verify the staged account

I created EMP006 with activate=false. The execution returned an immutable Okta account ID, re-queried the target and confirmed STAGED status. It also reported the PostgreSQL account record and success audit committed. Saving the stable target ID allowed future actions to address the same account even if its profile changed.

Step 6 Rerun the completed creation

I reran the EMP006 creation and received ALREADY_COMPLETE with no account creation or database writes. This demonstrated idempotent processing: repeating completed work did not create another account.

[SCREENSHOT NEEDED: Sanitized EMP006 creation execution and safe rerun showing employeeNumber, activate=false, direct target verification, STAGED and database commit; hide live login domains and target account IDs.]

Future-start deferral and duplicate-source handling need separate tests. Skipping an already-completed account does not verify how the workflow handles a duplicate source identity.

Controlled activation and pending setup

Step 7 Review activation before applying it

I kept creation and activation separate. The activation dry run checked the active HR record, saved account, creation audit, protected-account boundaries and direct target state. It proposed sendEmail=false and reported zero target modifications and zero PostgreSQL writes.

Step 8 Request activation and verify target status

I applied activation for EMP006. The execution recorded a durable intent, verified PROVISIONED on the target and reported the saved status and audit committed. It also reported activation material stored privately outside Git. Public captures omit private file locations and activation material.

I reran EMP006 activation after the confirmed result. It reported activation already confirmed and performed no activation or database writes.

Observed stage	Meaning in this lab	Evidence boundary
STAGED	Account created without activating it	Target status verified after creation
PROVISIONED	Activation request processed; setup still pending	Execution confirms this status
Pending user action	Okta interface shows incomplete user setup	People and profile captures agree
ACTIVE and sign-in	Setup completed and authentication demonstrated	Not established by this evidence

[SCREENSHOT NEEDED: Sanitized activation execution and rerun retaining sendEmail=false, PROVISIONED and commit results; remove account IDs, private activation-material paths and any activation secrets.]

[SCREENSHOT NEEDED: Okta People or synthetic-user profile page showing pending user action; remove live email domains, tenant identifiers and browser details.]

PROVISIONED meant the connector action had completed while user setup was still pending. I did not treat that result as proof that the users could sign in.

Step 9 Evaluate the required access

I evaluated EMP006 against the managed entitlement catalog. An entitlement is an access grant, such as a group or role. Birthright access follows baseline eligibility, while department and job-title rules determine additional access.

Source condition	Required group	Access basis
employeeType = FTE	Group-All-Employees	Birthright
department = Security	Group-Security-Department	Attribute based
jobTitle = Security Analyst	Group-Security-Tools	Role based

Step 10 Review the group assignment dry run

I checked identity and account correlation, the PROVISIONED target status, the catalog and the target mappings. Figure 3 shows three proposed additions, with no target modifications or database writes.

```
(.venv) iga@iga-engine-01:~/iga-lab$ python -m tests.test_okta_entitlements_dry_run
JBC IGA — EMP006 ENTITLEMENT DRY RUN
Authoritative identity and account correlation: PASS
Verified account status: PROVISIONED
Policy, database catalog, and target group mappings: PASS

EXPECTED ACCESS
WOULD_ADD | Group-All-Employees
  Policy: {"employeeType": "FTE"}
WOULD_ADD | Group-Security-Department
  Policy: {"department": "Security"}
WOULD_ADD | Group-Security-Tools
  Policy: {"jobTitle": "Security Analyst"}

MANAGED ACCESS OUTSIDE POLICY
None

GROUPS OUTSIDE THE MANAGED CATALOG — LEAVE UNCHANGED
Everyone

POSTGRESQL ENTITLEMENT COMPARISON
Target membership without database record: None
Database record without managed target membership: None
Database records outside configured catalog: None

Policy-required entitlements: 3
Membership additions proposed: 3
Target modification requests: 0
PostgreSQL writes: 0
Authentication-only token POSTs are permitted.
ENTITLEMENT DRY-RUN VALIDATION PASSED
DRY RUN / NO OKTA MODIFICATIONS PERFORMED
(.venv) iga@iga-engine-01:~/iga-lab$
```

Figure 3. The policy dry run proposes the three required Security joiner groups and leaves the unmanaged Everyone group unchanged.

The comparison found no mismatch between managed target memberships and saved records at that point. Everyone was outside the managed catalog and remained unchanged, keeping the operation within the access the lab managed.

Group provisioning and failure handling

Step 11 Apply and verify the three assignments

I applied the EMP006 plan with a recorded intent for each group. The output reported three verified assignments, three saved PostgreSQL entitlement records and per-group success audits. I then reran the operation. It found all three groups already present and reported no membership or database writes.

[SCREENSHOT NEEDED: Tightly framed EMP006 assignment and safe-rerun output showing three verified memberships, three database records and no repeat writes; preserve the original result and hide sensitive identifiers.]

Step 12 Stop and reconcile an unconfirmed action

During a live group assignment, HTTP 403 stopped the operation. The connector retained its pending intent and instructed reconciliation before retrying. Later captures show success, but the exact corrective change was not documented.

This distinguished an intended change, an API response and confirmed target state. Durable intents and target re-query checks made a partial action reviewable. The DPoP nonce challenge in Step 1 was a specific protocol exchange; it did not justify blindly retrying authorization failures.

Step 13 Create the remaining managed accounts

I ran bulk creation for the five remaining identities. The captures show all six accounts verified in STAGED status and recorded in PostgreSQL. A rerun found zero missing accounts and reported no writes.

Step 14 Activate the remaining staged accounts

After the EMP006 checks, I requested activation for the other five STAGED accounts. The bulk run verified all six in PROVISIONED status; its rerun proposed zero requests. The commands reported no activation emails sent.

Bulk assignment and target comparison

Step 15 Complete the bulk membership comparison

I extended group assignment to the lab population. Figure 4 starts with six existing memberships and five proposed additions. After execution, it reports eleven verified memberships across six identities, with saved PostgreSQL records and success audits. The run sent no activation or membership-removal requests.

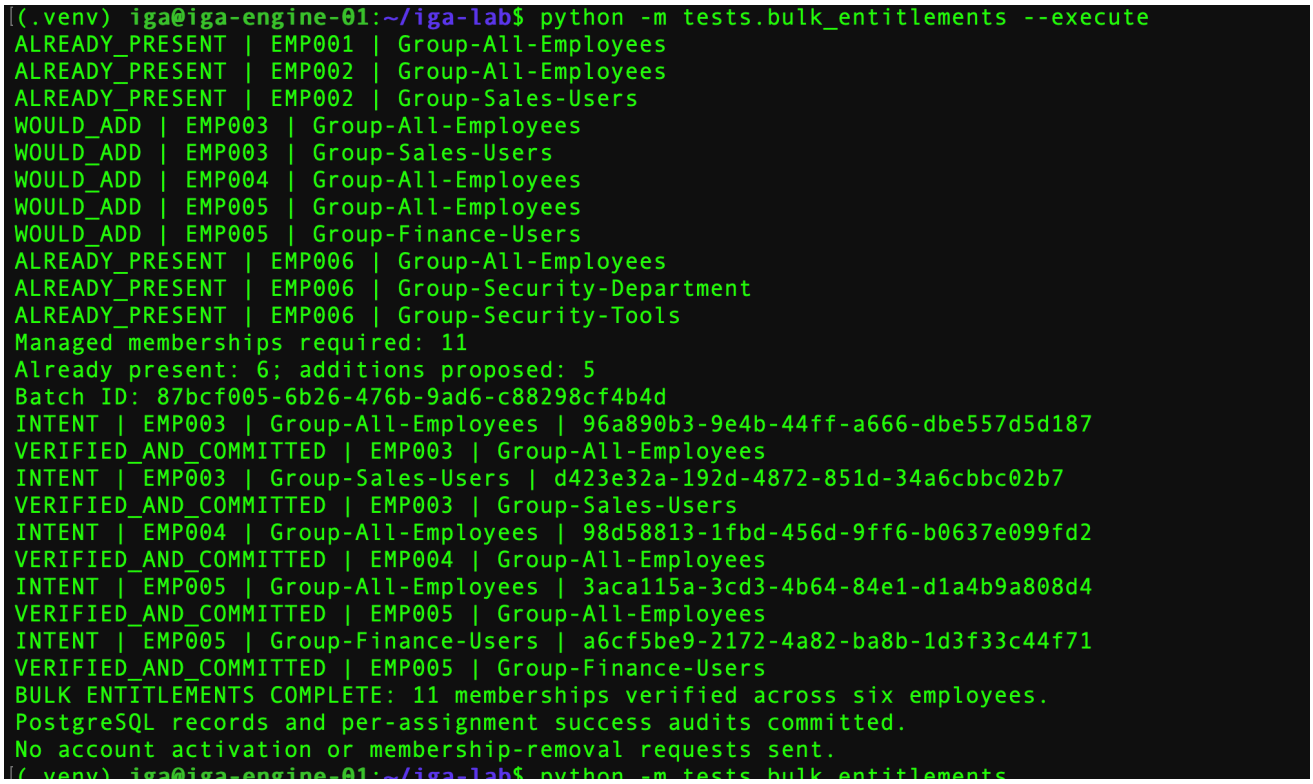


Figure 4. The bulk run fills five missing assignments and verifies eleven managed memberships across six synthetic identities.

The managed distribution was one group each for EMP001 and EMP004, two each for Sales identities EMP002 and EMP003, two for Finance identity EMP005, and three for Security identity EMP006. Target Groups tabs corroborated Sales and Finance membership. Those captures need public replacements because they show live email domains.

[SCREENSHOT NEEDED: Sanitized target Groups page showing the synthetic joiner's three managed assignments; retain Everyone to show the managed boundary and hide live domains and target identifiers.]

Step 16 Check safety tests and final results

I tested the protected-account boundaries shown in Figure 5. The administrator was blocked, an unprotected account was permitted and an empty account ID was rejected. The saved creation-related suite reported ten tests OK; the group-assignment suite reported eleven tests OK. These counts describe the captured runs.

```
(.venv) iga@iga-engine-01:~/iga-lab$ python -m tests.test_okta_protection
================================================
JBC IGA - PROTECTED ACCOUNT TEST
================================================
Protected-account configuration: PASS
Protected administrator blocked: PASS
Unprotected account permitted: PASS
Empty account ID rejected: PASS

ALL PROTECTION TESTS PASSED
(.venv) iga@iga-engine-01:~/iga-lab$
```

Figure 5. The protected-account checks block the administrator and reject an empty account identifier.

Control or result	Captured evidence	Status
Connector reads and token cache	E014, E016	Tested
EMP006 creation and safe rerun	E030, E031	Verified target and saved result
Activation and pending setup	E002, E003, E001, E035	Verified PROVISIONED; setup pending
Three EMP006 group assignments	E004, E005	Verified execution and rerun
Bulk eleven managed memberships	E007, E036, E037	Verified execution; UI corroboration
Wrong person or group blocked; 403 not retried	E017	Captured tests passed

Result and remaining work

The milestone established controlled account creation, activation requests and group provisioning with identity correlation, target verification and safe successful reruns. Six accounts were managed, user setup remained pending and eleven managed memberships were verified.

Remaining checks include final connector privileges, sanitized creation and activation captures, independent dry-run state comparisons, user sign-in, future-start handling and missing-manager handling. Approval workflows, Entra integration, downstream REST provisioning and its failure recovery remain outside the verified result.